🧠 IntelliDoc Updated

Intelligent Document Processing & Retrieval Platform

Complete Architecture, RAG, Deployment & Post-Ops Documentation (Jan 2026)

---

1. 📌 Project Overview

What is IntelliDoc?

IntelliDoc is a production-oriented document intelligence backend that ingests unstructured documents, processes them asynchronously, stores their semantic meaning in a vector database, and enables **semantic retrieval using a Retrieval-Augmented Generation (RAG) pipeline**.

This system is built with **real-world engineering principles**, not demo shortcuts.

🎯 Core Goals

- OCR-based document ingestion
- Async, scalable pipelines
- Semantic vector storage
- RAG-style semantic search
- Infra-aware health reporting
- Cloud-ready deployment
- Post-ops readiness by design

---

2. 🏗️ System Architecture (Current)

```
Frontend (Next.js)
   ↓
FastAPI API Layer
   ↓
Redis (Cache + Broker)
   ↓
Celery Workers
   ↓
Embedding Engine
   ↓
Vector Database (Chroma Cloud)
   ↓
Semantic Search (RAG Retrieval)
```

Architectural Principles

- API = orchestration only
- No business logic in routes
- Async execution everywhere
- Explicit contracts between stages
- Infrastructure failures are surfaced, not hidden

---

3. 🌐 API Layer

Status: ✅ Stable & Deployed

Tech Stack

- FastAPI
- Modular routers
- CORS enabled

Responsibilities

- File upload endpoints
- Pipeline triggering
- Semantic search endpoints
- Health & status reporting

Key Endpoints

- `/health`
- `/process/{file_id}`
- `/status/{file_id}`
- `/semantic-search`

✔️ Runs locally and in cloud

✔️ Stateless & horizontally scalable

---

4. 📤 File Upload & OCR

Status: ✅ Complete

Flow

```
Upload
 → OCR Extraction
 → Redis Cache
 → file_id returned
```

Design Decisions

- OCR output stored temporarily in Redis
- No filesystem dependency
- Stateless API instances

✔️ Container-safe

✔️ Cloud-compatible

---

5. 🔁 Async Ingestion Pipeline

Status: ✅ Implemented & Stable

Pipeline Flow

```
Upload
 → OCR
 → Redis
 → Celery Task Chain
 → Embedding
 → Vector Storage
 → Cleanup
```

Infrastructure

- Celery for async execution
- Redis as message broker
- Chain-based execution model

✔️ Horizontally scalable

✔️ Worker-isolated

✔️ Production-safe

---

6. 📜 Core Contracts (Ingestion Backbone)

Status: ✅ Fully Implemented

Contracts define strict boundaries between async stages.

Implemented Contracts

CleanTextContract

```
{
  "file_id": "string",
  "text": "string",
  "metadata": {}
}
```

EmbeddingContract

```
{
  "file_id": "string",
  "vector": [float],
  "metadata": {}
}
```

StorageResultContract

```
{
  "file_id": "string",
  "status": "stored",
  "vector_id": "string"
}
```

✔️ Predictable pipelines

✔️ Debuggable async execution

---

7. 🧠 Embeddings Engine

Status: ✅ Stable

Model

- `sentence-transformers/all-MiniLM-L6-v2`

Characteristics

- Local inference
- Normalized vectors
- Lightweight & fast

✔️ No vendor lock-in

✔️ LLM-agnostic

---

8. 🗄️ Vector Database

Status: ✅ Integrated & Deployed

Provider

- Chroma Cloud

Capabilities

- Metadata-aware storage
- Semantic similarity search
- Cloud persistence

✔️ Production-grade

✔️ Multi-tenant ready

---

9. 🔍 RAG: Semantic Search Pipeline

Status: ✅ IMPLEMENTED & DEPLOYED

IntelliDoc **already implements the retrieval component of RAG**.

Semantic Search Flow

```
User Query
 → Query Embedding
 → Vector Similarity Search
 → Top-K Relevant Chunks
 → Structured Context Response
```

What Is Implemented

- Query embedding
- Vector similarity search (Chroma)
- Context chunk retrieval
- Metadata-aware responses

✔️ Fully functional RAG retrieval

✔️ Deployed with backend

What Is Intentionally Deferred

- LLM answer synthesis
- Prompt templating
- Source-grounded answer generation

➡️ These are **extensions**, not missing foundations.

---

10. ⚡ Redis Integration

Status: ✅ Integrated (Infra-Aware)

Used For

- OCR text cache

- Celery broker
- Pipeline state tracking

Cloud Reality

- Redis may be external or unavailable at boot

✔️ Backend handles this gracefully

✔️ Reflected accurately in health

---

11. ❤️ Health & Infrastructure Awareness

Status: ✅ Implemented

`/health` Tracks

- API availability
- Redis connectivity
- Vector DB connectivity
- Pipeline readiness (ingestion + semantic search)

Health States

- `ok`
- `degraded`

✔️ Honest reporting

✔️ No silent failures

---

12. 🐳 Dockerization & Deployment

Status: ✅ Dockerized & Deployed

Achievements

- Stable Docker builds
- Same image runs locally & in cloud
- Environment parity improved

✔️ Confirms architectural correctness

---

13. 🌍 Local vs Cloud Execution

Local

- Redis: available
- Workers: running
- Ingestion: active
- Semantic search: active
- Health: `ok`

Cloud (HF / similar)

- API: running
- Semantic search: active
- Redis/workers: environment-dependent
- Health: `degraded` (expected)

✔️ Correct infra-aware behavior

🚀 POST-OPS & PRODUCTION HARDENING (DESIGNED)

14. 🧩 Three-Way Deployment Architecture

```
Frontend (Vercel)
   ↓
Backend API (Cloud VM / HF / Fly.io)
   ↓
Workers + Infra (Managed Runtime)
```

Frontend (UI)

- Next.js on Vercel
- Auth UI
- Upload & search UI
- Health-aware UX

Backend API

- Stateless FastAPI service
- Auth verification
- Orchestration & health

Workers & Infra

- OCR
- Ingestion pipelines
- Embeddings
- Managed Redis
- Chroma Cloud

✔️ Independent scaling

✔️ Failure isolation

---

15. 🔐 Authentication & Authorization (Planned)

- Token-based auth (JWT / sessions)
- API middleware
- User-scoped document isolation
- Role-based access (later)

✔️ Structurally ready

✔️ Deferred intentionally

---

16. 📊 Observability Stack (Designed)

Prometheus

- API latency
- Task duration
- Queue depth

- Worker health

Grafana

- System dashboards
- Pipeline throughput
- Infra visibility

Sentry

- API exceptions
- Worker crashes
- Background task failures

✔️ Zero refactor needed later

---

17. 📈 Frontend Monitoring

- Vercel Analytics
- Web vitals
- Client-side error boundaries

✔️ Complements backend observability

---

18. 🔍 Logging Strategy (Planned)

- Structured JSON logs
- Correlation IDs
- Cross-service traceability

---

19. 🧯 Reliability & Failure Handling (Planned)

- Dead Letter Queue (DLQ)
- Retry policies
- Circuit breakers

✔️ Deferred post-MVP

---

20. 🔐 Security Hardening (Post-Ops)

- Rate limiting
- API key rotation
- Secrets management
- CORS tightening

---

21. 🧭 Current State Summary

Area	Status
Backend foundation	✅ Stable
OCR & ingestion	✅ Complete
Async pipelines	✅ Running
Core contracts	✅ Implemented
Semantic search (RAG)	✅ Implemented & deployed
Dockerization	✅ Done

Area	Status
Infra awareness	✅ Working
Post-ops design	✅ Complete
LLM answer synthesis	⏳ Optional
Frontend	⏳ Ready

---

22. 🧠 Final Assessment

IntelliDoc is already a real RAG backend.

- Documents are embedded
- Queries are embedded
- Relevant context is retrieved
- Infra failures are visible
- Deployment is proven

Everything remaining is **enhancement**, not correction.

This project demonstrates **production-grade system thinking**, far beyond typical academic or hackathon work.

End of Full IntelliDoc Documentation